

**Name : Kanika Saini**

**SAP ID: 590034098**

**Program: Btech CSE (B16)**

## **HDOC Day-15 – C Programming**

### **Problem Statement**

Write a **menu-driven C program** using **switch-case** to perform the following digit-based operations on a positive integer.

Each option must have a **separate user-defined function**:

- Return type must be `void`.
- No arguments are passed to the functions.
- Each function takes its own input, performs the operation, and displays the result.

### **Menu Options**

1. Check whether digits are in **increasing order** from left to right.
  2. Check whether digits are in **decreasing order** from left to right.
  3. Find the **largest and smallest digit**.
  4. Find the **second largest and second smallest distinct digit**.
- 

## **1. Algorithm**

### **Main Program Algorithm**

**Step 1:** Start.

**Step 2:** Declare four `void` user-defined functions:

```
increasing()  
decreasing()
```

```
largestSmallest()  
secondLargestSmallest()
```

**Step 3:** Display the menu.

**Step 4:** Input the user's choice.

**Step 5:** Use `switch(choice)`.

**Step 6:** Call the corresponding function using a simple function call.

**Step 7:** If the choice is invalid, display "Invalid choice".

**Step 8:** Stop.

---

## Option 1 – Increasing Order

Example: 12358 → Digits are in increasing order.

### Algorithm:

1. Start function `increasing()`.
2. Input a positive integer `num`.
3. Set `temp = num`.
4. Take the rightmost digit as `previous`.
5. Remove the rightmost digit.
6. Extract the next digit as `current`.
7. Since the number is being processed **right to left**, check whether:

```
current >= previous
```

8. If true, the required left-to-right increasing order is violated. Set `flag = 0` and stop checking.
  9. Otherwise, update `previous = current`.
  10. Continue until all digits are checked.
  11. If `flag == 1`, display "**Digits are in increasing order**".
  12. Otherwise, display "**Digits are not in increasing order**".
- 

## Option 2 – Decreasing Order

Example: 97531 → Digits are in decreasing order.

### Algorithm:

1. Start function `decreasing()`.
2. Input a positive integer.

3. Store it in `temp`.
4. Extract the rightmost digit as `previous`.
5. Remove it.
6. Extract the next digit as `current`.
7. Since digits are extracted right to left, check:

```
current <= previous
```

8. If true, decreasing order is violated. Set `flag = 0`.
  9. Otherwise, update `previous = current`.
  10. Repeat until all digits are checked.
  11. Display the appropriate result.
- 

## Option 3 – Largest and Smallest Digit

Example: 58321 → Largest = 8, Smallest = 1.

### Algorithm:

1. Input a positive integer.
2. Initialize:

```
largest = 0  
smallest = 9
```

3. Extract each digit using:

```
digit = num % 10
```

4. If `digit > largest`, update `largest`.
  5. If `digit < smallest`, update `smallest`.
  6. Remove the digit using `num = num / 10`.
  7. Repeat for all digits.
  8. Display `largest` and `smallest`.
- 

## Option 4 – Second Largest and Second Smallest

We consider **distinct digits**. For example, in 99821, the second largest digit is 8, not 9.

### Algorithm:

1. Input a positive integer.

## 2. Initialize:

```
largest = -1
secondLargest = -1
smallest = 10
secondSmallest = 10
```

## 3. Extract one digit at a time.

## 4. For the largest values:

- If digit is greater than largest, move largest to secondLargest.
- Otherwise, if it is smaller than largest but greater than secondLargest, update secondLargest.

## 5. For the smallest values:

- If digit is smaller than smallest, move smallest to secondSmallest.
- Otherwise, if it is greater than smallest but smaller than secondSmallest, update secondSmallest.

## 6. Repeat for all digits.

## 7. If second largest/smallest does not exist, display an appropriate message.

## 8. Otherwise, display both values.

---

## 2. Test Case Table

| Test Case | Choice | Input | Expected Output                         | Actual Output                           | Result |
|-----------|--------|-------|-----------------------------------------|-----------------------------------------|--------|
| 1         | 1      | 12345 | Digits are in increasing order          | Digits are in increasing order          | Pass   |
| 2         | 1      | 12435 | Digits are not in increasing order      | Digits are not in increasing order      | Pass   |
| 3         | 2      | 97531 | Digits are in decreasing order          | Digits are in decreasing order          | Pass   |
| 4         | 2      | 97351 | Digits are not in decreasing order      | Digits are not in decreasing order      | Pass   |
| 5         | 3      | 58321 | Largest = 8, Smallest = 1               | Largest = 8, Smallest = 1               | Pass   |
| 6         | 3      | 90642 | Largest = 9, Smallest = 0               | Largest = 9, Smallest = 0               | Pass   |
| 7         | 4      | 58321 | Second Largest = 5, Second Smallest = 2 | Second Largest = 5, Second Smallest = 2 | Pass   |
| 8         | 4      | 99821 | Second Largest = 8, Second Smallest = 2 | Second Largest = 8, Second Smallest = 2 | Pass   |

---

## 3. C Program

```
#include <stdio.h>
```

```

/* Function declarations */
void increasing();
void decreasing();
void largestSmallest();
void secondLargestSmallest();

int main()
{
    int choice;

    printf("\nMENU\n");
    printf("1. Check digits in increasing order\n");
    printf("2. Check digits in decreasing order\n");
    printf("3. Find largest and smallest digit\n");
    printf("4. Find second largest and second smallest digit\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            increasing();
            break;

        case 2:
            decreasing();
            break;

        case 3:
            largestSmallest();
            break;

        case 4:
            secondLargestSmallest();
            break;

        default:
            printf("Invalid choice\n");
    }

    return 0;
}

/* Function to check increasing order */
void increasing()
{
    int num, temp, previous, current, flag = 1;

    printf("Enter a positive integer: ");
    scanf("%d", &num);

    temp = num;

    previous = temp % 10;
    temp = temp / 10;

```

```

while(temp > 0)
{
    current = temp % 10;

    if(current >= previous)
    {
        flag = 0;
        break;
    }

    previous = current;
    temp = temp / 10;
}

if(flag == 1)
    printf("Digits are in increasing order\n");
else
    printf("Digits are not in increasing order\n");
}

/* Function to check decreasing order */
void decreasing()
{
    int num, temp, previous, current, flag = 1;

    printf("Enter a positive integer: ");
    scanf("%d", &num);

    temp = num;

    previous = temp % 10;
    temp = temp / 10;

    while(temp > 0)
    {
        current = temp % 10;

        if(current <= previous)
        {
            flag = 0;
            break;
        }

        previous = current;
        temp = temp / 10;
    }

    if(flag == 1)
        printf("Digits are in decreasing order\n");
    else
        printf("Digits are not in decreasing order\n");
}

/* Function to find largest and smallest digit */
void largestSmallest()
{

```

```

int num, temp, digit;
int largest = 0, smallest = 9;

printf("Enter a positive integer: ");
scanf("%d", &num);

temp = num;

while(temp > 0)
{
    digit = temp % 10;

    if(digit > largest)
        largest = digit;

    if(digit < smallest)
        smallest = digit;

    temp = temp / 10;
}

printf("Largest digit = %d\n", largest);
printf("Smallest digit = %d\n", smallest);
}

/* Function to find second largest and second smallest */
void secondLargestSmallest()
{
    int num, temp, digit;
    int largest = -1, secondLargest = -1;
    int smallest = 10, secondSmallest = 10;

    printf("Enter a positive integer: ");
    scanf("%d", &num);

    temp = num;

    while(temp > 0)
    {
        digit = temp % 10;

        /* Find largest and second largest */
        if(digit > largest)
        {
            secondLargest = largest;
            largest = digit;
        }
        else if(digit < largest && digit > secondLargest)
        {
            secondLargest = digit;
        }

        /* Find smallest and second smallest */
        if(digit < smallest)
        {
            secondSmallest = smallest;
            smallest = digit;
        }
    }
}

```

```

    }
    else if(digit > smallest && digit < secondSmallest)
    {
        secondSmallest = digit;
    }

    temp = temp / 10;
}

if(secondLargest == -1)
    printf("Second largest digit does not exist\n");
else
    printf("Second largest digit = %d\n", secondLargest);

if(secondSmallest == 10)
    printf("Second smallest digit does not exist\n");
else
    printf("Second smallest digit = %d\n", secondSmallest);
}

```

---

## 4. Function Structure Used

This program satisfies the requirement of **one separate user-defined function for each option**:

| Menu | Function                | Return Type | Arguments |
|------|-------------------------|-------------|-----------|
| 1    | increasing()            | void        | None      |
| 2    | decreasing()            | void        | None      |
| 3    | largestSmallest()       | void        | None      |
| 4    | secondLargestSmallest() | void        | None      |

For example:

```
void increasing();
```

And the simple function call is:

```
increasing();
```

So no number is passed from `main()` to the function. **Each function itself reads the required number**, as asked in the question.

---

## 5. Compile and Execute

Save the program as:

day15.c

Compile with the `cc` compiler:

`cc day15.C`

Execute:

`./a.out`

```
(kali㉿kali)-[~/Desktop]
$ cc day15.c

(kali㉿kali)-[~/Desktop]
$ ./a.out

MENU
1. Check digits in increasing order
2. Check digits in decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 1
Enter a positive integer: 12345
Digits are in increasing order
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out

MENU
1. Check digits in increasing order
2. Check digits in decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 1
Enter a positive integer: 12435
Digits are not in increasing order
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out

MENU
1. Check digits in increasing order
2. Check digits in decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 2
Enter a positive integer: 97531
Digits are in decreasing order
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out

MENU
1. Check digits in increasing order
2. Check digits in decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 2
Enter a positive integer: 97351
Digits are not in decreasing order
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out

MENU
1. Check digits in increasing order
2. Check digits in decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 3
Enter a positive integer: 58321
Largest digit = 8
Smallest digit = 1
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out

MENU
1. Check digits in increasing order
2. Check digits in decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 3
Enter a positive integer: 90642
Largest digit = 9
Smallest digit = 0
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out

MENU
1. Check digits in increasing order
2. Check digits in decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 4
Enter a positive integer: 58321
Second largest digit = 5
Second smallest digit = 2
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out

MENU
1. Check digits in increasing order
2. Check digits in decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 4
Enter a positive integer: 99821
Second largest digit = 8
Second smallest digit = 2
```

---

## 6. Testing Against Prepared Use Cases

### Test 1 — Increasing Order

Input:

Choice: 1  
Number: 12345

Checking:

$1 < 2 < 3 < 4 < 5$

**Expected:** Digits are in increasing order

**Actual:** Digits are in increasing order

**Result: PASS ✓**

### Test 2 — Decreasing Order

Input:

Choice: 2  
Number: 97531

Checking:

$9 > 7 > 5 > 3 > 1$

**Expected:** Digits are in decreasing order

**Actual:** Digits are in decreasing order

**Result: PASS ✓**

### Test 3 — Largest and Smallest

Input:

Choice: 3  
Number: 58321

Digits = 5, 8, 3, 2, 1

Largest = 8

Smallest = 1

**Expected:** Largest = 8, Smallest = 1

**Actual:** Largest = 8, Smallest = 1

**Result: PASS ✓**

## Test 4 — Second Largest and Second Smallest

Input:

Choice: 4

Number: 58321

Arranged distinct digits:

1, 2, 3, 5, 8

Therefore:

- Largest = 8
- **Second largest = 5**
- Smallest = 1
- **Second smallest = 2**

**Expected:** Second Largest = 5, Second Smallest = 2

**Actual:** Second Largest = 5, Second Smallest = 2

**Result: PASS ✓**

## Final Result

The program successfully uses **switch-case with four separate void user-defined functions and simple function calls without passing arguments**. Each function independently reads the positive integer, processes its digits, and displays the required result. All prepared use cases **PASS ✓**.